

Análise de Técnicas de Qualidade de Serviço para Transmissão de Áudio e Vídeo em tempo Real

Luiza Nunes, Nicolas Salles e Pietro Lessa

Repositório do Git: <https://github.com/pietrolessa/qos-mininet-smooth-streaming>

1. Introdução

Com o aumento do uso de aplicações que exigem baixa latência, como transmissões de vídeo e áudio em tempo real, torna-se cada vez mais importante o uso de técnicas de Qualidade de Serviço (QoS – *Quality of Service*) para garantir um bom desempenho da rede mesmo em situações de congestionamento. Neste trabalho, buscamos aplicar e analisar diferentes técnicas de QoS em um ambiente de rede simulado com o Mininet, focando na transmissão de mídia por meio do protocolo RTP (*Real-time Transport Protocol*). O principal objetivo é manter a qualidade do áudio e do vídeo mesmo quando a rede está sobrecarregada, melhorando a experiência do usuário final.

2. Objetivos

O objetivo principal deste estudo é avaliar como diferentes técnicas de QoS afetam a qualidade da transmissão de áudio e vídeo em redes congestionadas. Para isso, utilizamos o comando `tc`, disponível no Linux, para aplicar essas técnicas no ambiente de simulação. Além de testar cada técnica de forma isolada, também buscamos verificar se a combinação entre elas pode trazer melhores resultados em termos de fluidez e estabilidade da transmissão.

3. Ambiente Experimental

O ambiente utilizado foi configurado no Mininet, com uma topologia composta por quatro hosts conectados através de dois switches. A transmissão de mídia foi realizada do host `h1` para o `h2`, usando `ffmpeg` como ferramenta de envio e `ffplay` como receptor. Para simular o congestionamento da rede, foi gerado tráfego de fundo entre os hosts `h3` e `h4` com a ferramenta `iperf`. As técnicas de QoS foram aplicadas com o comando `tc`, e, em alguns momentos, o Wireshark foi utilizado para analisar o tráfego da rede de forma mais detalhada.

4. Metodologia

Os testes foram realizados em três etapas principais. Primeiro, observamos o comportamento da transmissão sem nenhuma técnica de QoS, para entender como o congestionamento afeta a qualidade da mídia. Em seguida, aplicamos as técnicas individualmente: HTB (*Hierarchical Token Bucket*), TBF (*Token Bucket Filter*) e PRIO (fila de prioridade). Por fim, combinamos essas técnicas para analisar se o uso conjunto poderia gerar resultados melhores.

Cada técnica tem uma função específica no controle do tráfego: o HTB é usado para reservar uma largura de banda mínima para um determinado fluxo; o PRIO permite que pacotes de maior prioridade sejam enviados primeiro; e o TBF ajuda a suavizar rajadas de tráfego, controlando o envio de pacotes com base em tokens. Durante os testes, observamos que, ao aplicar cada técnica separadamente, houve alguma melhoria na qualidade, mas os melhores resultados apareceram quando combinamos as três técnicas. Um ponto importante é que a combinação apenas de HTB com TBF, sem o PRIO, não foi suficiente para garantir uma boa reprodução do vídeo.

No caso da priorização do tráfego RTP, usamos o prio qdisc, enquanto o HTB foi configurado para garantir uma largura de banda mínima, e o TBF para controlar os picos de tráfego. A ideia foi reduzir a competição do tráfego de mídia com o tráfego de fundo, garantindo que os pacotes do vídeo e do áudio chegassem com mais consistência ao destino.

5. Resultados e Discussão

Sem as técnicas de QoS, a transmissão de vídeo e áudio apresentou vários problemas, como travamentos e perda de pacotes, principalmente quando o tráfego de fundo estava ativo. Quando aplicamos apenas o PRIO, percebemos uma leve melhora, com menos travamentos, mas ainda havia interrupções. A técnica HTB sozinha trouxe uma melhoria mais significativa, com uma reprodução mais contínua. No entanto, foi a combinação de HTB com TBF que garantiu os melhores resultados, mantendo a estabilidade da transmissão mesmo com a rede congestionada.

Durante os testes, encontramos um problema na aplicação das técnicas. Embora as configurações parecessem corretas, a priorização do tráfego RTP não estava funcionando como esperado. Investigando mais a fundo, descobrimos que o Mininet, ao configurar largura de banda com `TCLink(bw=10)`, já cria automaticamente uma estrutura de classes HTB com identificadores como 5: e 5:1. O erro estava em tentarmos aplicar as técnicas em uma nova hierarquia com identificadores diferentes (1:), que, na prática, não existiam na interface da rede.

A solução foi ajustar as configurações para utilizar a hierarquia que já existia. Após isso, a aplicação do PRIO e do TBF passou a funcionar corretamente sobre a classe 5:1. Com essa correção, a transmissão passou a ser mais estável, e o vídeo foi reproduzido sem travamentos, mesmo com o tráfego de fundo em execução. Esse diagnóstico mostrou que, além de escolher bem as técnicas de QoS, é importante entender como o Mininet organiza a rede por padrão para garantir que as configurações realmente sejam aplicadas.

6. Código-Fonte

O código utilizado foi escrito em Python, utilizando a API do Mininet para automatizar tanto a criação da topologia da rede quanto a aplicação das técnicas de QoS. O script também inclui comandos para gerar o tráfego de fundo com `iperf`, além de permitir a captura de pacotes com o Wireshark. O código foi organizado de forma modular e clara para facilitar a leitura, reutilização e adaptação para outros testes.

7. Conclusão

Os testes realizados mostram que as técnicas de QoS têm um papel fundamental na garantia da qualidade da transmissão de mídia em redes congestionadas. A combinação de HTB, TBF e PRIO foi a mais eficiente, garantindo uma reprodução contínua e sem perdas perceptíveis. Também foi possível perceber a importância de aplicar corretamente essas técnicas dentro da estrutura de rede criada automaticamente pelo Mininet. Para trabalhos futuros, seria interessante testar outras abordagens, como o uso de técnicas de gerenciamento ativo de filas (AQM – *Active Queue Management*), para ampliar ainda mais o controle sobre o congestionamento em redes com tráfego multimídia.

7.5 Diagnóstico e Correções

Durante a execução dos testes, percebemos que, mesmo com as configurações de QoS aparentemente corretas, o tráfego RTP não estava sendo priorizado como deveria. Depois de uma análise mais cuidadosa, descobrimos que o Mininet, ao usar TCLink(bw=10), já cria uma hierarquia HTB automaticamente, com identificadores específicos como 5: e 5:1. O erro estava em tentar aplicar as técnicas usando outra hierarquia (1:), que não existia.

A correção foi reaplicar as configurações de QoS respeitando a hierarquia existente, utilizando a classe 5:1. Após essa mudança, o comportamento da rede melhorou significativamente, com a priorização de pacotes funcionando como esperado. Essa etapa reforçou a importância de conhecer o funcionamento interno das ferramentas utilizadas, para evitar que configurações bem planejadas sejam ineficazes por estarem sendo aplicadas de forma incorreta.